

# **Smart Contract Assessment**

*Moonwell*

# **HALBORN**

# Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

## ABOUT THIS REPORT

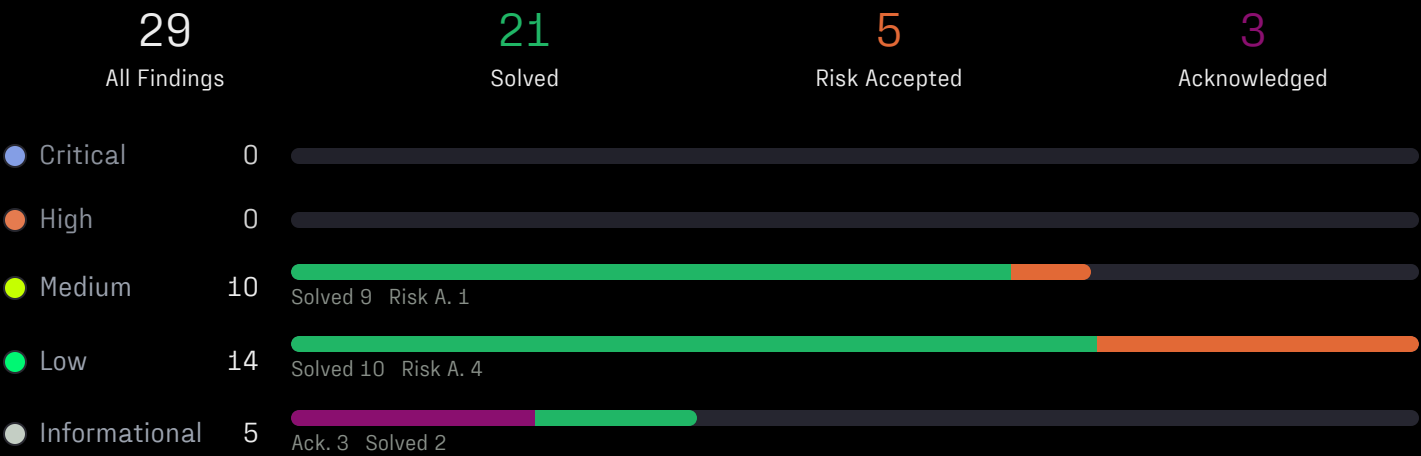
ASSESSMENT TYPE  
Assurance Assessment

SOURCE CODE  
 [mamo-contracts](#)

DATE OF ENGAGEMENT  
Jul 29, 2026 - Aug 10, 2026

ASSESSED COMMIT ID  
🔗 7470920

## SEVERITY BREAKDOWN



# INTRODUCTION

Moonwell engaged Halborn to perform a security assessment of their smart contracts starting on July 30th, 2026 and ending on August 10th, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The Mamo codebase in scope consists of smart contracts implementing a leveraged Aerodrome concentrated-liquidity yield strategy that borrows against USDC collateral on Moonwell to LP into a cbBTC/WETH pool, wrapped by a pooled vault and a per-user account layer that manage synchronous deposits, asynchronous redemptions, leverage and range adjustments, and management/performance fee crystallization.

# ASSESSMENT SUMMARY

Halborn was allocated 8 days for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in

blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, **Halborn** identified several areas for improvement to reduce the likelihood and impact of security risks, which were mostly addressed by the **Moonwell** team. The main recommendations were:

- In `_unwindLiquidity()`, whenever `remainingLiq == 0` after a removal, reset `$.tokenId = 0` (and `$.postTickLower/$.postTickUpper`) the same way `settleImpl()` does on a full settle, so a subsequent call correctly treats the book as flat instead of assuming an active staked position.
- Price the strategy's held AERO balance in `netEquityUsdc/nav()` using the AERO/USD Chainlink feed.
- When `supply == 0`, still advance `$.lastFeeAccrualTimestamp` before returning, so the fee clock consumes the dormancy window rather than deferring it onto the next depositor.

## **TEST APPROACH AND METHODOLOGY**

**Halborn** conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.

The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (**Foundry**).

- Fork testing against main networks ( **Foundry** ).
- Static security analysis of scoped contracts, and imported functions (Slither).

## **I Risk Methodology**

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

### **4.1 EXPLOITABILITY**

#### **ATTACK ORIGIN (AO):**

Captures whether the attack requires compromising a specific account.

#### **ATTACK COST (AC):**

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

#### **ATTACK COMPLEXITY (AX):**

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

**METRICS:**

| EXPLOITABILITY METRIC ( $m_e$ ) | METRIC VALUE     | NUMERICAL VALUE |
|---------------------------------|------------------|-----------------|
| Attack Origin (AO)              | Arbitrary (AO:A) | 1               |
|                                 | Specific (AO:S)  | 0.2             |
| Attack Cost (AC)                | Low (AC:L)       | 1               |
|                                 | Medium (AC:M)    | 0.67            |
|                                 | High (AC:H)      | 0.33            |
| Attack Complexity (AX)          | Low (AX:L)       | 1               |
|                                 | Medium (AX:M)    | 0.67            |
|                                 | High (AX:H)      | 0.33            |

Exploitability  $E$  is calculated using the following formula:

$$E = \prod m_e$$

**4.2 IMPACT**

**CONFIDENTIALITY (C):**

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

**INTEGRITY (I):**

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

**AVAILABILITY (A):**

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

**DEPOSIT (D):**

Measures the impact to the deposits made to the contract by either users or owners.

**YIELD (Y):**

Measures the impact to the yield generated by the contract for either users or owners.

**METRICS:**

| IMPACT METRIC ( $m_I$ ) | METRIC VALUE   | NUMERICAL VALUE |
|-------------------------|----------------|-----------------|
| Confidentiality (C)     | None (C:N)     | 0               |
|                         | Low (C:L)      | 0.25            |
|                         | Medium (C:M)   | 0.5             |
|                         | High (C:H)     | 0.75            |
|                         | Critical (C:C) | 1               |
| Integrity (I)           | None (I:N)     | 0               |
|                         | Low (I:L)      | 0.25            |
|                         | Medium (I:M)   | 0.5             |
|                         | High (I:H)     | 0.75            |
|                         | Critical (I:C) | 1               |
| Availability (A)        | None (A:N)     | 0               |
|                         | Low (A:L)      | 0.25            |
|                         | Medium (A:M)   | 0.5             |
|                         | High (A:H)     | 0.75            |
|                         | Critical (A:C) | 1               |
| Deposit (D)             | None (D:N)     | 0               |
|                         | Low (D:L)      | 0.25            |
|                         | Medium (D:M)   | 0.5             |
|                         | High (D:H)     | 0.75            |
|                         | Critical (D:C) | 1               |
| Yield (Y)               | None (Y:N)     | 0               |
|                         | Low (Y:L)      | 0.25            |
|                         | Medium (Y:M)   | 0.5             |
|                         | High (Y:H)     | 0.75            |

| IMPACT METRIC ( $m_I$ ) | METRIC VALUE   | NUMERICAL VALUE |
|-------------------------|----------------|-----------------|
|                         | Critical (Y:C) | 1               |

Impact  $I$  is calculated using the following formula:

$$I = \max(m_I) + \frac{\sum m_I - \max(m_I)}{4}$$

### 4.3 SEVERITY COEFFICIENT

#### REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

#### SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

#### METRICS:

| SEVERITY COEFFICIENT ( $C$ ) | COEFFICIENT VALUE | NUMERICAL VALUE |
|------------------------------|-------------------|-----------------|
| Reversibility ( $r$ )        | None (R:N)        | 1               |
|                              | Partial (R:P)     | 0.5             |
|                              | Full (R:F)        | 0.25            |
| Scope ( $s$ )                | Changed (S:C)     | 1.25            |
|                              | Unchanged (S:U)   | 1               |

Severity Coefficient  $C$  is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score  $S$  is obtained by:

$$S = \min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

|          |      |        |     |               |
|----------|------|--------|-----|---------------|
| Critical | High | Medium | Low | Informational |
|----------|------|--------|-----|---------------|

9 - 10

7 - 8.9

4.5 - 6.9

2 - 4.4

0 - 1.9

## Scope

### REPOSITORY



(a) Repository:  **mamo-contracts**

(b) Assessed Commit ID:  7470920d2508505407bb50da6daf5d0249fd4c94

(c) Items in scope:

- ✓  src 5 files
  - ✓  leveraged-aero 3 files
    -  LeveragedAerodromeCLStrategy.sol Solidity
    -  LeveragedAeroFees.sol Solidity
    -  LeveragedAeroManager.sol Solidity
    -  LeveragedAeroVault.sol Solidity
    -  MamoLeveragedAeroStrategy.sol Solidity

**Out-of-Scope:** Third party dependencies and economic attacks.

### REMEDIATION COMMIT ID:



-  9cb52d6
-  043f281
-  669dd68
-  cc9b52f
-  5d01baf
-  575738d
-  6632bd8
-  90cbdb2
-  e7b7992
-  99afcd8
-  c675424
-  6f75498
-  b7f27eb
-  e8d80d7
-  2e4de48
-  eb129f4

✦ e52190d

✦ b9cad39

✦ dd68c1e

✦ f6849fc

**Out-of-Scope:** New features/implementations after the remediation commit IDs.

## Assessment Summary & Findings Overview

| #       | TITLE                                                                                                                | SEVERITY | SCORE | STATUS               |
|---------|----------------------------------------------------------------------------------------------------------------------|----------|-------|----------------------|
| HAL-001 | Missing tokenId reset after a full liquidity unwind permanently bricks settlement and rebalancing functions          | ● Medium | 5.0   | SOLVED<br>08/19/2026 |
| HAL-002 | nav() omits AERO rewards auto-claimed to the strategy between compounds                                              | ● Medium | 5.0   | SOLVED<br>08/19/2026 |
| HAL-003 | Phantom management fee minted against a depositor who reopens an empty book                                          | ● Medium | 5.0   | SOLVED<br>08/19/2026 |
| HAL-004 | _redeemCoverShortfall reverts on partial idle USDC, breaking both the redeem deadman and the lever-down safety valve | ● Medium | 5.0   | SOLVED<br>08/19/2026 |
| HAL-005 | Rerange consumes stayers' idle USDC into the new position in asset-mode, breaking the principal-conserved invariant  | ● Medium | 5.0   | SOLVED<br>08/19/2026 |
| HAL-006 | Rerange cannot recentre a one-sided position, forcing a full settle-and-redeploy cycle                               | ● Medium | 5.0   | SOLVED<br>08/19/2026 |
| HAL-007 | _settleShortfall passes an unclamped repay sentinel                                                                  | ● Medium | 5.0   | SOLVED               |

| #       | TITLE                                                                                                                                   | SEVERITY | SCORE | STATUS                      |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------|----------|-------|-----------------------------|
|         | while approving only the swap output, reverting every settlement path when the swap fills below debt                                    |          |       | 08/19/2026                  |
| HAL-008 | _settleShortfall bounds swap output to maxSlippageBps but hardcodes the spend budget at 110% of fair, permitting systematic overpayment | ● Medium | 5.0   | SOLVED<br>08/19/2026        |
| HAL-009 | Permissionless dust deposits force HWM ratchets, converting NAV volatility into performance fees                                        | ● Medium | 5.0   | RISK ACCEPTED<br>08/19/2026 |
| HAL-010 | _rebalanceCover sells the entire surplus-leg balance, silently flipping the fund's directional exposure on every lever-down             | ● Medium | 5.0   | SOLVED<br>08/19/2026        |
| HAL-011 | Idle USDC ambiguity between fulfilled withdrawal proceeds and re-deposit funds can cause withdrawal re-lock                             | ● Low    | 2.5   | SOLVED<br>08/19/2026        |
| HAL-012 | _settleShortfall() sizes repay budget from stale borrowBalanceStored(), risking MoonwellRedeemFailed and a bricked settlement           | ● Low    | 2.5   | SOLVED<br>08/19/2026        |
| HAL-013 | Permissionless initialize() in BaseStrategy enables proposer-role hijacking via a two-step clone-then-initialize race                   | ● Low    | 2.5   | SOLVED<br>08/19/2026        |
| HAL-014 | redeemUnwindImpl() sweeps residual cbBTC/WETH to USDC with minOut=0, unlike                                                             | ● Low    | 2.5   | SOLVED<br>08/19/2026        |

| #       | TITLE                                                                                                                         | SEVERITY | SCORE | STATUS                      |
|---------|-------------------------------------------------------------------------------------------------------------------------------|----------|-------|-----------------------------|
|         | the oracle-priced minimum used in settleImpl()                                                                                |          |       |                             |
| HAL-015 | _redeemCollateral() uses stale exchangeRateStored(), systematically understating a redeemer's pro-rata USDC share             | ● Low    | 2.5   | SOLVED<br>08/19/2026        |
| HAL-016 | Redeemers can exit at pre-fee prices while deposits are closed, dropping the deferred performance fee onto stayers            | ● Low    | 2.5   | SOLVED<br>08/19/2026        |
| HAL-017 | Strategy becomes permanently un-leverageable once tokenId reaches 0 in Executed state, forcing full settlement + redeployment | ● Low    | 2.5   | RISK ACCEPTED<br>08/19/2026 |
| HAL-018 | Asymmetric virtual-offset between deposit and redeem deviates from ERC-4626 canonical accounting                              | ● Low    | 2.5   | RISK ACCEPTED<br>08/19/2026 |
| HAL-019 | Full-redeem branch passes an unbounded amountInMax to _redeemCoverShortfall, exposing the last redeemer to MEV extraction     | ● Low    | 2.5   | RISK ACCEPTED<br>08/19/2026 |
| HAL-020 | DegenerateRange revert in deployIdleImpl cascades into compound, bricking harvest and interest hedging during volatility      | ● Low    | 2.5   | RISK ACCEPTED<br>08/19/2026 |
| HAL-021 | AERO harvest slippage floor in compoundImpl ignores the USDC/USD peg feed                                                     | ● Low    | 2.5   | SOLVED<br>08/19/2026        |
| HAL-022 | settleImpl sells both LP legs but never liquidates the claimed reward token,                                                  | ● Low    | 2.5   | SOLVED<br>08/19/2026        |

| #       | TITLE                                                                                                                                               | SEVERITY        | SCORE | STATUS                     |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|-------|----------------------------|
|         | stranding it outside every holder's redemption basis                                                                                                |                 |       |                            |
| HAL-023 | emergencyRedeem reads Chainlink through _settleShortfall, bricking the deadman in the exact outage it exists to survive                             | ● Low           | 2.5   | SOLVED<br>08/19/2026       |
| HAL-024 | Donating a single share to the vault permanently disables the asset rescue safety valve                                                             | ● Low           | 2.5   | SOLVED<br>08/19/2026       |
| HAL-025 | Single shared maxDelay staleness threshold used across price feeds of differing volatility                                                          | ● Informational | 1.7   | ACKNOWLEDGED<br>08/19/2026 |
| HAL-026 | Proposer role in BaseStrategy is set permanently at initialize() with no rotation mechanism                                                         | ● Informational | 1.7   | SOLVED<br>08/19/2026       |
| HAL-027 | Discarded Comptroller error code on the before getAccountLiquidity() snapshot in deleverageImpl()                                                   | ● Informational | 1.7   | ACKNOWLEDGED<br>08/19/2026 |
| HAL-028 | settleImpl() unconditionally requires all three Chainlink feeds to be live, blocking settlement during an oracle outage even when no swap is needed | ● Informational | 1.7   | ACKNOWLEDGED<br>08/19/2026 |
| HAL-029 | Missing bps cap in performanceFeeShares underflows and silently defers all fee crystallisation, enabling an owner-triggered fee-shift window        | ● Informational | 0.5   | SOLVED<br>08/19/2026       |

## Findings & Tech Details

HAL-001

SOLVED

### Missing tokenId reset after a full liquidity unwind permanently bricks settlement and rebalancing functions

• Medium · 5.0 A0:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

#### DESCRIPTION

The `_unwindLiquidity(uint256 num, uint256 den)` helper in `src/leveraged-aero/LeveragedAeroManager.sol` is the shared routine every position-touching function uses to unstake the CL position NFT from the Aerodrome gauge, remove a `num/den` fraction of its liquidity, and, only when liquidity remains afterward, restake it:

SOL 1135-1145

```
1135 function _unwindLiquidity(uint256 num, uint256 den) private {
1136     Layout storage $ = _layout();
1137     uint256 tokenId_ = $.tokenId;
1138     if (tokenId_ == 0) return; // flat book - no LP to unwind
1139     ...
1140     ICLGauge(gauge_).withdraw(tokenId_);
1141     uint128 liqToRemove = (num == den) ? liq : uint128(Math.mulDiv(uint256(liq), num,
den));
1142     if (liqToRemove > 0) { ...decreaseLiquidity + collect... }
1143     (, uint128 remainingLiq) = _npmPositionData();
1144     if (remainingLiq > 0) _approveAndStake(gauge_, tokenId_); // restake ONLY if
liquidity remains
1145 }
```

However, when `num == den` (or the fraction otherwise rounds `liqToRemove` up to the position's full liquidity), `remainingLiq` is `0` and the restake branch is skipped, leaving the position NFT unstaked and empty. `_unwindLiquidity` never resets `$.tokenId` in this case; that cleanup only happens in `settleImpl()`, a code path this call never reaches. The result is an orphaned state: `$.tokenId != 0` (so every downstream function still treats the book as "has an active staked position") while the NFT is actually empty and no longer held by the gauge.

Every other function that touches the position starts by unconditionally calling `ICLGauge.withdraw(tokenId_)` to unstake it before modifying it, `settleImpl()`, `rerangeImpl()`, `_wrapAddRestake()` (shared by `deployIdleImpl()` and the lever-up branch of `adjustLeverage`), and `redeemUnwindImpl()` (the body of `requestRedeem` → `fulfillRedeem` / `emergencyRedeem`). `ICLGauge.withdraw` requires the gauge to currently hold the token staked to this contract; since it no longer does, every one of these calls reverts before ever reaching the logic that would clear

`tokenId` or otherwise recover. There is no code path in the contract that checks whether the token is still actually staked before calling `withdraw`, so once this state is reached, no function can move the position forward again.

The strategy is an EIP-1167 minimal-proxy clone with an immutable implementation (explicitly documented as non-upgradeable in `sherwood/BaseStrategy.sol`), and `LeveragedAeroVault._bind()` binds a strategy address once with no rebind function, so there is no on-chain path to patch or replace the broken instance in place. Fast `redeem()` (`LeveragedAerodromeCLStrategy.sol:1155`) is unaffected, it prices via `nav()` (which reads the NFT directly through `INonfungiblePositionManager.positions()`, not the gauge) and pays out via `fastRedeemImpl()` (which reads Moonwell collateral directly), so depositor principal remains withdrawable. Every other function that depends on `_unwindLiquidity` / `_wrapAddRestake`, however, is permanently dead for this vault instance.

Exploitation steps:

1. Proposer calls `LeveragedAerodromeCLStrategy.adjustLeverage(0, minLiq, minOut)`, a fully permitted call ( $0 \leq \text{maxLtvBps}$ ), driving `adjustLeverageImpl` to call `_leverDown(debtUsdc, debtUsdc, minOut)`, i.e. `num == den`.
2. `_unwindLiquidity` removes 100% of the position's liquidity, leaves the NFT unstaked, and does not reset `$.tokenId`. The call completes successfully, debt is repaid from the collected legs, collateral is untouched.
3. Any subsequent call to `settleStrategy()`, `rerange()`, `deployIdle()`, `compound()`, `requestRedeem` → `fulfillRedeem`, or `emergencyRedeem()` reverts inside `_unwindLiquidity` / `_wrapAddRestake` on the now-unconditional `gauge.withdraw(tokenId_)` call, because the gauge no longer holds that token staked.
4. The vault permanently loses all rebalancing, compounding, and settlement capability; since the implementation is an immutable clone and the vault cannot rebind to a new strategy, this cannot be fixed in place, only a fresh strategy/vault deployment with a migrated user base can restore functionality.

The same orphaning is also reachable without any privileged role via the permissionless `deleverage()` safety valve (`LeveragedAerodromeCLStrategy`, intentionally not `onlyProposer`) whenever its health-recovery unwind rounds `liqToRemove` up to the position's full liquidity, e.g. when the position is deeply unhealthy and needs a near-total unwind to restore `minHealthBps`.

#### RECOMMENDATION

In `_unwindLiquidity()`, whenever `remainingLiq == 0` after a removal, reset `$.tokenId = 0` (and `$.postTickLower` / `$.postTickUpper`) the same way `settleImpl()` does on a full settle, so a

subsequent call correctly treats the book as flat instead of assuming an active staked position.

#### REMEDIATION COMMENT

Addressed in: [🔗 9cb52d6](#)

**SOLVED:** The **Moonwell** team solved the issue in the specified commit id by following the mentioned recommendation.

---

## nav() omits AERO rewards auto-claimed to the strategy between compounds

• Medium • 5.0

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N

### DESCRIPTION

`LeveragedAerodromeCLStrategy.nav()` is the sole price oracle for the vault, every `deposit()`, `redeem()`, async redeem, and `compound()`'s own fee crystallization prices against it. For an active position it delegates to `LeveragedAeroValuation.netEquityUsdc()`, whose documented formula (`LeveragedAeroValuation.sol:44`) is `idleStrategy + collateral + clLegs + idleLegs - debt`, nowhere does this read `IERC20(aero).balanceOf(strategy)`:

### SOL

```
function netEquityUsdc(...) internal view returns (uint256) {
    uint256 idleStrategy = IERC20(usdc).balanceOf(strategy);
    uint256 collateral    = _collateralUsdc(mToken, strategy);
    (uint256 clLegs, uint256 idleLegs) = _legsUsdc(...);
    uint256 debt          = _debtUsdc(usdcBorrow, strategy);
    // AERO on the strategy from prior gauge.withdraw() auto-claims is NOT included
    return idleStrategy + collateral + clLegs + idleLegs - debt;
}
```

However, `ICLGauge.withdraw()` "collects outstanding rewards automatically on withdrawal" (`ISlipstream.sol:178-180`), and every position-touching op, `_unwindLiquidity` (`LeveragedAeroManager.sol`), `_wrapAddRestake` (`LeveragedAeroManager.sol`), calls it unconditionally as part of its normal flow (settle, rerange, deployIdle, adjustLeverage/deleverage all unstake-then-restake). Each of those auto-claims any pending AERO to the strategy's own token balance as a side effect.

Only `compoundImpl` (`LeveragedAeroManager.sol:436-483`) ever reads that AERO balance and swaps it into USDC/NAV. Between claim and the next `compound()`, real AERO sits on the contract, publicly observable (`AERO.balanceOf(strategy)`), and entirely unpriced.

The mispricing is cadence-limited in the steady state (frequent compounds keep the balance small), but if `compound()` is paused, reverts (e.g. under fee-misconfig conditions), or is simply infrequent, the unpriced AERO balance can build up materially. Direction: NAV under-reports then deposits mint more shares than fair value → depositors capture some of that value from existing stakers once the AERO is eventually swapped and folded into NAV by a later `compound()`.

Exploitation requires no privileged role, `compound()` is `onlyProposer`, but any EOA can observe `AERO.balanceOf(strategy)` (public state) and time a `deposit()` to land while a large unclaimed balance is sitting there, then `redeem()` shortly after the next `compound()` realizes it, capturing a share of value that accrued before they were invested.

Compounding the same root cause, pending gauge rewards, AERO still accruing inside the gauge and not yet claimed to the strategy, are also never read by `nav()`, `netEquityUsdc()` never calls `ICLGauge.earned(strategy)`. Value earned since the last claim lives inside the gauge, not on the strategy, so it's absent from every term of the NAV formula. Unlike the `balanceOf(strategy)` case, which only manifests after a position op auto-claims, this gauge-side omission is present continuously in steady state from the moment the position is staked, widening the exploitable mispricing window from "briefly after position ops" to "always open."

#### RECOMMENDATION

Price the strategy's held AERO balance in `netEquityUsdc / nav()` using the same AERO/USD Chainlink feed already read in `compoundImpl`'s oracle-floor check (`Math.mulDiv(aeroBal, _readUsd8($.aeroUsdFeed), 1e20)`), so no window exists where claimed-but-unswapped rewards are invisible to pricing.

#### REMEDIATION COMMENT

Addressed in: [🔗 043f281](#)

**SOLVED:** The `Moonwell` team solved the issue in the specified commit id by following the mentioned recommendation.

---

# Phantom management fee minted against a depositor who reopens an empty book

• Medium • 5.0

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N

## DESCRIPTION

`_crystallizeFees()` is the single fee-accrual point invoked pre-op by every share-issuing / share-burning entrypoint, `deposit()`, `redeem()`, `fulfillRedeem()`, `emergencyRedeem()`, `compound()`. It reads `$.lastFeeAccrualTimestamp`, hands `(navPre, supply, hwm, lastAccrual, nowTs, mgmtBps, perfBps, protocolBps)` to the pure crystallize library function, and applies the returned `(feeShares, newHwm, newLast, protocolUsdc)` to storage. Crystallize computes the management-fee slice from `dt = nowTs - lastAccrual` and unconditionally returns `newLast = nowTs`, so `dt` correctly represents "elapsed time since the LAST crystallise, regardless of supply changes in between."

However, the wrapper short-circuits before the library ever runs:

SOL 791-800

```
791 function _crystallizeFees(uint256 navPre) private {
792     Layout storage $ = _layout();
793     uint256 supply = IERC20(vault()).totalSupply();
794     if (supply == 0) return;
795     if ($.lastFeeAccrualTimestamp == 0) {
796         $.lastFeeAccrualTimestamp = block.timestamp;
797         return;
798     }
799     ...
800 }
```

The `supply == 0 → return` guard is intended as a cheap no-op (no supply, no shares to dilute, so the library would produce `feeShares = 0` anyway). What it silently omits is the `$.lastFeeAccrualTimestamp = block.timestamp` write that the library performs on the same path. As a result **any elapsed period during which the vault holds no capital is not consumed by the accrual clock**, it is retained and charged to whoever next re-opens the book. On the next crystallise-triggering op (a deposit, redeem, or compound after the book is reopened), `dt` spans the entire dormant window, and crystallize mints management-fee shares against a supply that only just came back into existence. **The depositor bore no risk during the dormancy window, yet is diluted for it.**

#### RECOMMENDATION

When `supply == 0`, still advance `$.lastFeeAccrualTimestamp` before returning, so the fee clock consumes the dormancy window rather than deferring it onto the next depositor.

#### REMEDIATION COMMENT

Addressed in: [🔗 669dd68](#)

**SOLVED:** The [Moonwell](#) team solved the issue in the specified commit id by following the mentioned recommendation.

---

HAL-004

SOLVED

## **\_redeemCoverShortfall reverts on partial idle USDC, breaking both the redeem deadman and the lever-down safety valve**

• Medium · 5.0 A0:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

### DESCRIPTION

`LeveragedAeroManager._redeemCoverShortfall()` is the shared primitive that swaps idle USDC into a deficit borrow token via Slipstream's exact-output swap. Its two guards disagree about what an under-budgeted cover means:

SOL 1368-1371

```
1368 | uint256 usdcBal = IERC20($.usdc).balanceOf(address(this));
1369 | if (usdcBal == 0 || amountOut == 0) return; // ← graceful for
    | empty
1370 | uint256 maxIn = usdcBal < amountInMax ? usdcBal : amountInMax;
1371 | LeveragedAeroValuation.swapExactOut(..., amountOut, maxIn); // ← REVERTS for 0 <
    | idle < needed
```

An empty balance returns gracefully through the explicit guard, and a balance at or above the pool-derived required input completes the swap normally. However, a balance between the two reverts: `maxIn` resolves to the full idle amount, which is less than the input the pool needs to deliver `amountOut`, and `exactOutputSingle` fails on `amountIn > amountInMaximum`. The primitive is therefore **non-monotone in idle USDC**, adding a small non-zero balance to a strategy that would have no-opped cleanly on empty converts the graceful path into a hard revert, violating the property that more resources should never produce a worse outcome. Two upstream call sites inherit that revert, each breaking its own contract.

**Site 1 - the full-redeem branch.** Reached via `emergencyRedeem()` → `_proportionalRedeem()` → `redeemUnwindImpl()`. The branch comment documents the intended design: Phase 1 either succeeds when idle covers the shortfall or no-ops when idle is zero, and Phase 2's `_settleShortfall()` mops up any residual debt from mUSDC collateral. In reality Phase 2 only runs if Phase 1 returns. Inside the partial-idle band Phase 1 reverts on the `amountInMax` breach, Phase 2 never executes, and the entire `emergencyRedeem()` reverts. The deadman is unreachable in precisely the "some idle, but not enough" corner, the holder's request stays open with no on-chain trustless exit until the idle balance grows through an external top-up or the impermanent loss unwinds naturally.

Site 2 - lever-down. Reached via `adjustLeverageImpl()` (proposer) or the permissionless `deleverageImpl()` → `_leverDown()` → `_rebalanceCover()` with an oracle-derived `buyMax`. The trigger is a dual shortfall, where both legs are short after `_redeemRepayFromCollected()`: each `_rebalanceCover()` invocation finds the sibling surplus leg empty so `_sweepLegToUsdc()` no-ops, and both fall back onto idle USDC. The same partial-idle revert follows. Recovery is easier here, the operator can top up USDC and retry, and the fund's health is unchanged by the revert, but the manager documents its accepted residuals elsewhere (the stored-index note, the H3 best-effort fee mint, the loud-diagnosable-failure comment on `InsufficientIdleForLeverUp`) and `_rebalanceCover()` carries no matching disposition for this case.

The caller at Site 1 is the request owner and at Site 2 the proposer or any EOA once health drops; no attacker is required, since the triggering state arises from ordinary operation. Small non-zero idle balances are common between deploys, and impermanent loss shortfalls are the expected condition the entire cover infrastructure exists to handle.

#### RECOMMENDATION

Pick one of the two behaviours and apply it consistently. Dropping the `usdcBal == 0` early return at `LeveragedAeroManager` makes both regimes revert loudly on an under-budgeted shortfall cover, matching the paired lever-up branch's `InsufficientIdleForLeverUp` philosophy and the codebase's stated preference for diagnosable failure over silent partial fills. Alternatively add a one-line comment at the early return in `_redeemCoverShortfall()` documenting the chosen policy, mirroring the style used for the other accepted residuals in the manager.

#### REMEDIATION COMMENT

Addressed in: [♣ cc9b52f](#)

**SOLVED:** The `Moonwell` team solved the issue in the specified commit id by adding a `bestEffort` flag to `swapExactOut` that catches the router revert and reports `filled=false`.

---

HAL-005

SOLVED

## Rerange consumes stayers' idle USDC into the new position in asset-mode, breaking the principal-conserved invariant

• Medium · 5.0 A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N

### DESCRIPTION

`LeveragedAeroManager.rerangeImpl()` sizes the new position by reading the strategy's raw token balances after unwinding the old NFT:

SOL 1085-1086

```
1085 | _amounts01(IERC20($.cbBTC).balanceOf(address(this)), // <-- reads USDC (asset-mode  
1086 | collapse)  
      | IERC20($.weth).balanceOf(address(this)))
```

In two-borrowed-legs mode this is correct, since `IERC20($.cbBTC).balanceOf()` returns an actual cbBTC balance that can only have come from the unwind. However, in asset-mode, where `legBIsAsset == true` and therefore `$.cbBTC == $.usdc`, the leg-B slot collapses onto the unit of account, and that same call returns the strategy's **entire** USDC balance. That balance bundles together three distinct pools of capital: the LP principal and fees just collected from the unwound position, pre-existing idle USDC from depositors that has not yet been passed to `deployIdle()`, and USDC set aside from prior reranges' collected residuals.

`_mintPosition(amt0, amt1, ...)` then consumes from that bundled figure at whatever ratio the new range demands, with no distinction between capital that belonged to the old position and capital that never did. The comment at [:534](#) claims that with no swap, principal is conserved, and that what remains is the remainder of one borrowed leg. In asset-mode that claim does not hold, the "remainder" and the "consumed" live in the same balance, so the one-borrowed-leg framing is meaningless when leg B is USDC.

### RECOMMENDATION

Bound the USDC handed to `_mintPosition()` in asset-mode to exactly what the unwind produced, rather than to the strategy's total balance. Snapshot the USDC balance before calling `_unwindLiquidity()`, take the difference afterwards, and pass that delta as the leg-B side of `_amounts01()`.

#### REMEDIATION COMMENT

Addressed in: [🔗 5d01baf](#)

**SOLVED:** The **Moonwell** team solved the issue in the specified commit id by following the mentioned recommendation.

---

HAL-006

SOLVED

## Rerange cannot recentre a one-sided position, forcing a full settle-and-redeploy cycle

● Medium · 5.0

A0:A/AC:L/AX:L/R:P/S:U/C:N/A:C/I:N/D:N/Y:N

### DESCRIPTION

`LeveragedAeroManager.rerangeImpl()` unwinds the old position with `_unwindLiquidity(1, 1)`, derives a fresh range from `centeredTickRange()` around the current tick, and mints into it via `_mintPosition(amt0, amt1, tickLower, tickUpper)`. The desired amounts passed to the mint are whatever the unwind collected, and `depositMins()` derives `amt0Min` and `amt1Min` from the new range at the current `sqrtp`.

However, the two are computed from different premises. The collected amounts reflect where the *old* range sat relative to price at unwind time, while the minimums reflect what a *newly centred* range requires. When the pool tick had already left `[tL, tU]` before the rerange, the old position is entirely one-sided and the collect returns only one token. The new range, centred on the current tick, brackets price and therefore wants both. With one desired amount at zero,

`LiquidityAmounts.getLiquidityForAmounts()` resolves to zero or degenerate liquidity, and the mint either reverts against a positive minimum on the empty side.

### RECOMMENDATION

Allow rerange to mint a one-sided position when only one side is available. Slipstream and Uniswap-V3-alike pools accept a mint whose range sits entirely to one side of the current tick, so rather than centring unconditionally, `rerangeImpl()` should detect that one collected amount is zero and select a range on the populated side, leaving `depositMins()` to compute a zero minimum for the absent token instead of a positive one. The position then reopens immediately in a valid state, and a subsequent rerange once price re-enters the band, or once the position accumulates the other side through trading, can recentre normally.

### REMEDIATION COMMENT

Addressed in: [🔗 575738d](#)

**SOLVED:** The [Moonwell](#) team solved the issue in the specified commit id by following the mentioned recommendation.

---

HAL-007

SOLVED

## **`_settleShortfall` passes an unclamped repay sentinel while approving only the swap output, reverting every settlement path when the swap fills below debt**

• Medium · 5.0

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

### DESCRIPTION

`LeveragedAeroManager._settleShortfall()` covers residual debt left over after `_settleRepayDebts()` by swapping USDC into the debt token through `_swapUsdcExactIn()`, then repaying that leg on Moonwell. The swap's `minAmtOut` is `debt × (10000 - maxSlippageBps) / 10000`, which is strictly less than the debt whenever `maxSlippageBps > 0`, that slack is the entire purpose of the floor. After the swap the function approves the amount actually received and then repays with the max sentinel:

SOL 1150-1157

```

1150  if (cbDebtRem > 0) {
1151      _swapUsdcExactIn($.cbBTC, cbUsdcNeed, cbDebtRem * (10000 - slip) / 10000); // ←
      min-out < debt
1152      uint256 cbBal2 = IERC20($.cbBTC).balanceOf(address(this));
1153      if (cbBal2 > 0) {
1154          IERC20($.cbBTC).forceApprove($.mCbBTC, cbBal2); // ←
      approves swap output only
1155      _repay($.mCbBTC, type(uint256).max); // ←
      Moonwell reads as "repay ALL"
1156      }
1157  }
```

However, under Moonwell and Compound-v2 semantics `repayBorrow(type(uint256).max)` is a sentinel meaning repay the full outstanding `accountBorrows`. The market resolves the sentinel to the full debt and calls `transferFrom(strategy, market, accountBorrows)`, pulling the debt rather than the approved balance. When the swap filled below the debt, the case the floor explicitly permits, the transfer exceeds both the allowance and the balance, and the entire transaction reverts.

Both sibling repay sites in the same file clamp correctly. `_settleRepayDebts()` passes `cbBal >= cbDebt ? type(uint256).max : cbBal`, using the sentinel only when the balance covers the debt. `_redeemCoverShortfall()` follows the same shape with `tokenBal >= amountOut ? amountOut : tokenBal`. `_repay()` itself is a thin wrapper over `repayBorrow()` and performs no clamping, leaving that responsibility to the caller, which every caller except this one discharges. The WETH leg at repeats the identical pattern.

#### RECOMMENDATION

Clamp both repay calls to the balance actually received, passing the max sentinel only when that balance covers the residual debt and the received amount otherwise.

#### REMEDIATION COMMENT

Addressed in: [🔗 6632bd8](#)

**SOLVED:** The [Moonwell](#) team solved the issue in the specified commit id by following the mentioned recommendation.

---

## **`_settleShortfall` bounds swap output to `maxSlippageBps` but hardcodes the spend budget at 110% of fair, permitting systematic overpayment**

• Medium · 5.0 A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:M/Y:N

### DESCRIPTION

`LeveragedAeroManager._settleShortfall()` sets two independent bounds on the cover swap, and only one of them tracks the operator's configured slippage tolerance. The spend budget at uses a hardcoded literal:

SOL 1140-1141

```
1140 | uint256 cbUsdcNeed    = _tokenToUsdc(cbDebtRem,    $.cbBTCDecimals, pBTC, pUsdc) * 11000 /
      | 10000;
1141 | uint256 wethUsdcNeed = _tokenToUsdc(wethDebtRem, $.wethDecimals, pETH, pUsdc) * 11000 /
      | 10000;
```

while the output floor scales correctly with configuration:

SOL 1152

```
1152 | _swapUsdcExactIn($.cbBTC, cbUsdcNeed, cbDebtRem * (10000 - slip) / 10000);
```

However, because the two bounds are structurally decoupled, a pool can be driven to the corner where both pass simultaneously, spending the full  $1.10 \times \text{fair}$  while delivering only  $(1 - \text{slip}) \times \text{debt}$ . Letting  $p$  be the oracle price of the debt token in USDC, the spend cap is  $1.10 \times p \times \text{debt}$  and the output floor is  $(1 - \text{slip}/10000) \times \text{debt}$ , so the effective price paid per unit of output is bounded only by  $p \times 1.10 / (1 - \text{slip}/10000)$ . At `maxSlippageBps = 100` that is roughly  $p \times 1.111$ , meaning the swap tolerates paying about 11.1% over oracle fair, some eleven times the declared tolerance. At `300` it is about 13.4%, and at the `1000` initialisation ceiling about 22.2%. In every configuration the tolerance is approximately  $10\% + \text{slip}\%$ , with the hardcoded buffer supplying the dominant term.

This is the only cover site in the file that bounds output quantity while letting spend float independently. `_rebalanceCover()` anchors its spend at `_tokenToUsdc(shortAmt, ...) × (10000 + maxSlippageBps) / 10000`, and step 5 of `settleImpl()` anchors the sell side at `fair × (10000`

`- slip) / 10000`. The team documented the `_rebalanceCover()` bound explicitly as the H1 mitigation, noting that a sandwiched permissionless deleverage cannot overpay past the oracle and slippage bound. The same reasoning applies here, and no NatSpec on `_settleShortfall()` justifies the divergence, this reads as a site the H1 sweep missed rather than a deliberate design choice.

#### RECOMMENDATION

Bound the spend to `maxSlippageBps` rather than to a hardcoded buffer, replacing the `11000 / 10000` multiplier on both `cbUsdcNeed` and `wethUsdcNeed` with `(10000 + slip) / 10000` and hoisting the `slip` read above the budget computation.

#### REMEDIATION COMMENT

Addressed in: [🔗 90cddb2](#)

**SOLVED:** The `Moonwell` team solved the issue in the specified commit id by following the mentioned recommendation.

---

# Permissionless dust deposits force HWM ratchets, converting NAV volatility into performance fees

● Medium · 5.0

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N

## DESCRIPTION

`LeveragedAerodromeCLStrategy.deposit()` crystallises fees against the pre-deposit NAV before pulling any USDC, routing through `_crystallizeAndNet()` → `_crystallizeBestEffort()` → `crystallizeFeesSelf()` → `_crystallizeFees()`. The entrypoint carries no access modifier beyond `nonReentrant`, no `onlyProposer`, no allowlist and enforces no minimum deposit size.

The only amount check is `if (shares == 0) revert ZeroShares();`, which a 1-wei deposit satisfies in any normally funded vault, since the virtual offset makes `mulDiv(1, supply + 1e6, navNet + 1)` strictly positive. Meanwhile `LeveragedAeroFees.performanceFeeShares()` treats the high-water mark as a one-way ratchet: it assigns `newHwmPerShareX = navPerShareX` whenever NAV per share exceeds the stored mark and refuses to lower it on drawdowns.

`_crystallizeFees()` then writes `$.hwmPerShare = newHwm` unconditionally, with no minimum interval between crystallisations and no dust filter anywhere in `src/leveraged-aero/`.

However, a discretely sampled high-water-mark fee is only fair when the fund controls the sampling schedule. Here the schedule is externally controllable: the entrypoint is permissionless and unrated, the crystallisation runs unconditionally, and the amount is unbounded below. **Any EOA can therefore choose the sampling moments** for roughly 1 wei of USDC plus gas, and the `feeRecipient` can do the same to farm its own fee.

## RECOMMENDATION

Impose a minimum interval on the performance and HWM portion of the crystallisation. Inside `_crystallizeFees()`, skip the HWM update when `block.timestamp - $.lastFeeAccrualTimestamp` falls below a configured floor of, say, six to twenty-four hours.

## REMEDIATION COMMENT

**RISK ACCEPTED:** The `Moonwell` team made a business decision to risk accept this finding and not alter the contracts, stating that:

The mechanism is real (the fee telescopes to the max sampled per-share, and sampling points are not all fund-controlled) but no caller profits — the ratchet dilutes everyone in favour of the fee recipient, and the peak can only be selected inside a calm, Chainlink-confirmed window, never

manufactured. The suggested minimum-interval gate was rejected as unsound: crystallising before every issuance/burn is the fairness invariant, the only available clock is advanced by every op (the gate would disable the perf leg outright), and an un-sampled window is a genuine fee escape for exits. The posture is now documented in the fee library header with a sampler-pays anchor test; `setOpenDeposits(false)` remains the operational lever.

---

## **`_rebalanceCover` sells the entire surplus-leg balance, silently flipping the fund's directional exposure on every lever-down**

• Medium · 5.0

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N

### DESCRIPTION

`LeveragedAeroManager._rebalanceCover()` is the imbalance-cleanup helper invoked at the tail of `_leverDown()` whenever the legs collected from a partial CL unwind do not pro-rata match the debt being repaid. It sells the over-collected surplus leg for USDC and uses the proceeds to buy exactly the deficit the other leg needs. However, the two sides of the swap are sized asymmetrically: solidity

SOL 774-777

```
774 | _sweepLegToUsdc(surplusTok, 0, minUsdcOut);           // ← SELL: keep = 0, sells WHOLE
    | balance
775 | ...
776 | uint256 buyMax = _tokenToUsdc(shortAmt, ...) * (10000 + slip) / 10000;
777 | _redeemCoverShortfall(deficitTok, deficitMkt, shortAmt, buyMax); // ← BUY: need-sized
    | against shortAmt
```

`_sweepLegToUsdc()` resolves `keep = 0` to "sell the entire strategy balance in this token" rather than only the surplus this unwind produced. When the strategy already holds an idle balance in that leg, a rerange remainder, which is a normal NAV-neutral state produced when a prior rerange could not consume the whole collected leg into the new range's ratio, that pre-existing balance is sold alongside the surplus.

The correct pattern exists elsewhere in the same file. `redeemUnwindImpl()` snapshots stayers' `(1-f)` share through `_stayerLeg()` before the unwind and passes that snapshot as `keep` to `_sweepLegToUsdc()`. `_rebalanceCover()` was left without the mirror.

The consequence is that `adjustLeverage()` and `deleverage()`, which are contractually leverage-only operations that should scale the delta profile by `(1-f)` and never change its sign, silently convert an on-book position leg into cash. The unwind and repay themselves preserve the fund's net exposure correctly, both the LP holding and the debt shrink by the same fraction, leaving the long or short position proportionally intact. It is the unbounded sweep that follows which breaks it: by selling the rerange remainder along with the genuine surplus, the operation strips exposure that was never part of the leverage reduction, and a book that was net long on that leg before the call can end up net

short after it. The entrypoint promises to reduce leverage by a fraction; the execution reduces leverage by that fraction and re-hedges the delta.

#### RECOMMENDATION

Bound the sell to what is actually needed to cover `shortAmt` rather than the whole balance.

#### REMEDIATION COMMENT

Addressed in: [🔗 5d01baf](#)

**SOLVED:** The `Moonwell` team solved the issue in the specified commit id by following the mentioned recommendation.

---

HAL-011

SOLVED

## Idle USDC ambiguity between fulfilled withdrawal proceeds and re-deposit funds can cause withdrawal re-lock

● Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

### DESCRIPTION

`MamoLeveragedAeroStrategy::depositIdle()` and `MamoLeveragedAeroStrategy::claimWithdrawnUsdc()` both act on the exact same, unbucketed balance, `usdc.balanceOf(address(this))`, with no on-chain flag distinguishing "idle USDC awaiting re-deployment" from "idle USDC that is fulfilled-withdrawal proceeds awaiting owner claim." When `LeveragedAerodromeCLStrategy::fulfillRedeem()` settles a request, it transfers `assetsOut` straight to the account contract via `safeTransfer(recipient, assetsOut)` with no callback or state update, the USDC simply becomes part of the same idle balance that `depositIdle()` sweeps.

`depositIdle()` is callable by `owner()` OR the registry-configured backend address (`mamoStrategyRegistry.getBackendAddress()`), while `claimWithdrawnUsdc()` is `onlyOwner`. Because the two entry points read the identical balance and neither checks the other's intent, whichever one is called first consumes the funds for that purpose, there is no reservation or accounting layer preventing the backend from re-deploying USDC the owner intended to withdraw.

### RECOMMENDATION

Implement a two-bucket accounting system: add a `pendingWithdrawalUsdc` state variable that is incremented when `fulfillRedeem()` delivers proceeds and decremented on `claimWithdrawnUsdc()`. `depositIdle()` should then only sweep `usdc.balanceOf(address(this)) - pendingWithdrawalUsdc`, and `claimWithdrawnUsdc()` should only transfer up to `pendingWithdrawalUsdc`.

### REMEDIATION COMMENT

Addressed in: [✦ e7b7992](#)

**SOLVED:** The [Moonwell](#) team solved the issue in the specified commit id by adding a recipient field to `RedeemRequest` so `fulfillRedeem` pays the owner directly, eliminating unclaimed proceeds on the account entirely.

---

## **`_settleShortfall()` sizes repay budget from stale `borrowBalanceStored()`, risking `MoonwellRedeemFailed` and a bricked settlement**

• Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

### DESCRIPTION

`LeveragedAeroManager::_settleShortfall()` sizes the USDC budget for covering remaining Moonwell debt using `borrowBalanceStored()`, an un-accrued, potentially stale snapshot, rather than `borrowBalanceCurrent()`:

#### SOL

```
uint256 cbDebtRem = IMoonwellMarket($.mCbBTC).borrowBalanceStored(address(this));
uint256 wethDebtRem = IMoonwellMarket($.mWeth).borrowBalanceStored(address(this));
```

A flat 10% buffer is then applied: `cbUsdcNeed = _tokenToUsdc(cbDebtRem, ...) * 11000 / 10000`. This budget drives `_swapUsdcExactIn($.cbBTC, cbUsdcNeed, ...)`, whose output is then passed to `_repay(market, type(uint256).max)`. If interest accrued since the last market interaction pushes the true, current debt more than 10% above the stale `borrowBalanceStored()` reading, the swap acquires fewer tokens than the live debt requires, `_repay` clears only the acquired amount, and residual debt remains on the cToken. `settleImpl()`'s final step then unconditionally calls `ICToken($.mUsdc).redeem(mBal)` to reclaim all collateral, with no debt-is-zero guard beforehand, Moonwell's `comptroller.redeemAllowed` rejects a full collateral redemption while any borrow balance remains, so the call reverts and the whole settlement transaction (`revert MoonwellRedeemFailed(err)`), is bricked.

This same class of bug, sizing a repay/redeem budget from a stale, un-accrued `borrowBalanceStored()` read instead of `borrowBalanceCurrent()`, recurs in other functions in the same contract, since no `accrueInterest()` call exists anywhere to force freshness.

### RECOMMENDATION

Replace `borrowBalanceStored()` with `borrowBalanceCurrent()` (which internally calls `accrueInterest()`) at all affected settlement/repay-budgeting sites.

### REMEDIATION COMMENT

Addressed in: [🔗 99afcd8](#)

**SOLVED:** The `Moonwell` team solved the issue in the specified commit id by following the mentioned recommendation.



# Permissionless initialize() in BaseStrategy enables proposer-role hijacking via a two-step clone-then-initialize race

• Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

## DESCRIPTION

`BaseStrategy::initialize()` has no caller restriction beyond a boolean guard:

SOL 77-83

```
77 function initialize(address vault_, address proposer_, bytes calldata data) external {
78     if (_initialized) revert AlreadyInitialized();
79     if (vault_ == address(0)) revert ZeroAddress();
80     if (proposer_ == address(0)) revert ZeroAddress();
81     _initialized = true;
82     _vault = vault_;
83     _proposer = proposer_;
```

The constructor only locks the template itself (`constructor() { _initialized = true; }`), this never runs on an EIP-1167 clone, so every freshly deployed clone starts with `_initialized = false` and is callable by anyone. `LeveragedAeroVault::cloneAndBind()` closes this atomically, `Clones.clone(template)`, `IStrategy(clone).initialize(...)`, and `_bind(clone)` all happen in one `onlyOwner` call, so no window exists for that path. However, the vault also still exposes a separate, live entry point, `setStrategy()`:

SOL 164-166

```
164 function setStrategy(address strategy_) external onlyOwner {
165     _bind(strategy_);
166 }
```

and `_bind()` validates only `require(IStrategy(strategy_).vault() == address(this), "LAV: strategy not bound to this vault");` no check on `proposer()` or on how/when `initialize()` was called. If a clone is ever deployed via a standalone `Clones.clone(template)` and initialized in a separate later transaction, any address can front-run the mempool-visible `initialize()` call with the correct `vault_` and an attacker-controlled `proposer_`. `_bind`'s pointer-only check then passes, and `setStrategy` binds the attacker-initialized clone, handing the attacker the `onlyProposer` role

(e.g. `updateParams`, `compound`, `rerange`, `adjustLeverage`) over a strategy now controlling vault funds.

#### RECOMMENDATION

Add a proposer-ownership check to `_bind()` so it verifies the clone was initialized as expected, e.g. require the caller of `setStrategy()` to also pass the expected `proposer_` and compare it against `IStrategy(strategy_).proposer()`, reverting if they differ.

#### REMEDIATION COMMENT

Addressed in: [🔗 c675424](#)

**SOLVED:** The `Moonwell` team solved the issue in the specified commit id by following the mentioned recommendation.

---

## redeemUnwindImpl() sweeps residual cbBTC/WETH to USDC with minOut=0, unlike the oracle-priced minimum used in settleImpl()

• Low · 2.5 A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

### DESCRIPTION

`LeveragedAeroManager::redeemUnwindImpl()` sweeps residual borrowed token balances to USDC after unwinding CL liquidity and repaying debt, passing a literal zero minimum output:

SOL 355-356

```
355 | _sweepLegToUsdc($.cbBTC, stayersCb, 0);
356 | _sweepLegToUsdc($.weth, stayersWeth, 0);
```

with an inline comment acknowledging the choice: `"min-out=0; aggregate guard applies"`.

`_sweepLegToUsdc()` forwards `minOut` straight into `LeveragedAeroValuation.swapExactIn()`'s Slipstream router call:

SOL 488-494

```
488 | ICLSwapRouter(router).exactInputSingle(
489 |     ICLSwapRouter.ExactInputSingleParams({
490 |         ...
491 |         amountOutMinimum: minOut,
492 |         sqrtPriceLimitX96: 0
493 |     })
494 | );
```

So `minOut=0` becomes `amountOutMinimum: 0` with `sqrtPriceLimitX96: 0`, no per-swap price floor and no price-limit backstop either. This is inconsistent with the identical sweep inside

`settleImpl()`, which computes an oracle-derived minimum before sweeping:

`_swapTokenToUsdc($.weth, _tokenToUsdc(balance, $.wethDecimals, pETH, pUsdc) * (10000 - slip) / 10000)`. Grepping every `_sweepLegToUsdc` call site confirms `minOut=0` is isolated to `redeemUnwindImpl`, `_rebalanceCover` passes an oracle-floor-derived nonzero minimum, and `settleImpl`'s wrapper does the same. The developer's own comment on `_rebalanceCover` confirms the oracle-floor guard built for `deleverage()`'s sweep is explicitly bypassed on the redeem path.

The aggregate guard the inline comment refers to is

`LeveragedAerodromeCLStrategy._proportionalRedeem`'s check `if (assetsOut < minOut)`

`revert InsufficientAssetsOut();`, using the redeem request's stored `minAssetsOut`. For `fulfillRedeem()` (`onlyProposer`), that stored value is set at request-creation time and is not refreshed before the swaps execute. So between request creation and fulfillment, a market mover or MEV searcher can sandwich either `exactInputSingle` call, since neither has its own floor, and extract value up to the margin between the current price and the stale `minAssetsOut` bound, before the aggregate check finally catches an outright bad total. `emergencyRedeem`'s per-request `minAssetsOut` is supplied fresh at call time, so it is not exposed to the same staleness window.

#### RECOMMENDATION

Compute an oracle-derived per-swap minimum for the two `_sweepLegToUsdc` calls in `redeemUnwindImpl()`, mirroring `settleImpl()`'s pattern (`_tokenToUsdc(balance, decimals, price, pUsdc) * (10000 - slip) / 10000`), so each swap is protected independently rather than relying solely on a request-time aggregate bound that can go stale for up to the `FULFILL_WINDOW`.

#### REMEDIATION COMMENT

Addressed in: [🔗 6f75498](#)

**SOLVED:** The `Moonwell` team solved the issue in the specified commit id by a proposer-supplied fresh floor at fulfil time, and the per-swap floors do exist but are fail-open by design.

---

## **`_redeemCollateral()` uses stale `exchangeRateStored()`, systematically understating a redeemer's pro-rata USDC share**

• Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

### DESCRIPTION

`LeveragedAeroManager::_redeemCollateral()` computes a redeeming holder's share of the strategy's mUSDC-denominated collateral using a cached, non-accrued exchange rate:

SOL 1388-1397

```
1388 function _redeemCollateral(uint256 shares, uint256 supply) private {
1389     address mUsdc_ = _layout().mUsdc;
1390     uint256 cBal = ICToken(mUsdc_).balanceOf(address(this));
1391     if (cBal == 0) return;
1392     uint256 rate = ICToken(mUsdc_).exchangeRateStored();
1393     uint256 totalUnderlying = (cBal * rate) / 1e18;
1394     uint256 toRedeem = Math.mulDiv(totalUnderlying, shares, supply);
1395     if (toRedeem == 0) return;
1396     _redeemUnderlying(mUsdc_, toRedeem);
1397 }
```

Since `exchangeRateStored()` is always  $\leq$  the true current rate (Moonwell's exchange rate only accrues upward between explicit `accrueInterest()` calls), `totalUnderlying`, and therefore `toRedeem` via `Math.mulDiv(totalUnderlying, shares, supply)`, is systematically understated whenever interest has accrued since the mUSDC market was last touched. There is no post-hoc reconciliation against an actual balance afterward, so the shortfall is not corrected; it silently accrues to the benefit of the remaining stayers rather than the redeeming holder. This is not, however, a case of the "correct" function being structurally unreachable: `_redeemCollateral()` carries no `view`/`pure` modifier and is called from `redeemUnwindImpl()`, itself a state-changing function inside a real redeem transaction, so nothing prevents forcing an accrual here. The actual reason `exchangeRateCurrent()` isn't used is that this codebase's `ICToken` interface only declares `exchangeRateStored()` in the first place, `exchangeRateCurrent()` was never wired in as an interface method, so it isn't a deliberate view-context tradeoff, just a narrower interface than the one Moonwell actually exposes.

The staleness is inconsistently neutralized depending on which redeem path runs: in the full-redeem branch, `_settleShortfall()` sometimes calls `_redeemUnderlying($.mUsdc, totalNeed)` before `_redeemCollateral()` runs, and that Moonwell `redeemUnderlying` call accrues interest as a side effect, making the subsequent `exchangeRateStored()` read effectively current, but only when

`_settleShortfall` doesn't early-return (i.e., only when `cbDebtRem` / `wethDebtRem` are nonzero). In the common case where idle USDC already covers the shortfall, or in the **partial-redeem branch**, where `_redeemCollateral()` is called directly at with no preceding repay/settle step, there is no same-transaction accrual at all, and the stale rate is used as-is. The magnitude of the shortfall is bounded by however much interest has accrued since the mUSDC market's last touch, typically small but it is a systematic, one-directional leak on every affected redemption.

The same class of stale-index leak exists in other parts of the contract, and treating `_redeemCollateral()` in isolation understates the scope. The pattern, read a Compound-style \*Stored accumulator, use its value as an absolute amount rather than as one side of a self-cancelling ratio, and consume it without a preceding accrual on the same market.

#### RECOMMENDATION

Call `ICToken(mUsdc_).accrueInterest()` (already exposed on the underlying Moonwell cToken, just not currently declared on the local `ICToken` interface) immediately before reading `exchangeRateStored()` in `_redeemCollateral()`, guaranteeing the rate reflects the current block regardless of which redeem path invoked it.

#### REMEDIATION COMMENT

Addressed in: [🔗 b7f27eb](#)

**SOLVED:** The **Moonwell** team solved the issue in the specified commit id by following the mentioned recommendation.

---

# Redeemers can exit at pre-fee prices while deposits are closed, dropping the deferred performance fee onto stayers

• Low · 2.5 A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

## DESCRIPTION

`strategyMint()` is gated on `depositsOpen`, but `strategyBurn()` deliberately is not, the natspec justifies this as "exits must keep working during an incident." At the same time, `LeveragedAerodromeCLStrategy` uses the "best-effort crystallise" pattern (`_crystallizeBestEffort`) on every share-issuing / share-burning op: `nav()` is read outside a `try/catch` (fail-closed on the price), but the actual `_crystallizeFees` call is wrapped in `try this.crystallizeFeesSelf(navPre) {} catch { emit FeeCrystallizeDeferred(...); }`. The catch is intentionally narrow-in-scope but not narrow-in-selector: it swallows any revert coming out of the crystallise, including the `strategyMint-of-fee-shares` call at `_crystallizeFees`, `ISyndicateVault(vault()).strategyMint($.feeRecipient, feeShares)`, which reverts with "LAV: deposits closed" when `depositsOpen == false`.

The redeem fast path uses this asymmetry to leak the pending performance-fee accrual to whoever exits first while deposits are closed:

SOL 1175-1183

```
1175 uint256 navPre = nav();
1176 uint256 navNet = _crystallizeAndNet(navPre, OP_REDEEM); // crystallise reverts → catch
    → HWM + owed unchanged
1177 uint256 supply = IERC20(vault_).totalSupply();
1178 assetsOut = Math.mulDiv(shares, navNet, supply); // navNet == navPre → pre-fee
    pricing
1179 ...
1180 IERC20(vault_).safeTransferFrom(msg.sender, address(this), shares);
1181 LeveragedAeroManager.fastRedeemImpl(assetsOut, idleShare);
1182 IERC20(_layout().usdc).safeTransfer(msg.sender, assetsOut);
1183 ISyndicateVault(vault_).strategyBurn(shares); // succeeds – NOT gated on
    depositsOpen
```

The performance leg of `_crystallizeFees` only mints when there is a real gain above the HWM (`newHwm > oldHwm`), and the management leg mints against elapsed `dt`. In either case, `feeShares > 0` triggers the gated `strategyMint`, which reverts. The catch fires, `HWM` and `lastFeeAccrualTimestamp` are NOT advanced, `protocolFeeOwed` is NOT incremented, and `navNet = navPre - (owedNow - owedBefore) = navPre - 0 = navPre`. The redeemer's payout is priced on

the **FULL pre-fee NAV**, and the burn goes through unconditionally. The same mechanism applies to the async path ( `fulfillRedeem` / `emergencyRedeem` via `_proportionalRedeem`, which uses the same `_crystallizeBestEffort` ) and to `_dischargeRedeemSkim` (which pays 0 to the protocol when `owed == 0`, and `owed` never grew because the crystallise was caught).

#### RECOMMENDATION

Either gate the exit paths symmetrically with the fee-mint gate, or make the crystallise fail-closed on redeem when there is a pending non-zero `feeShares`.

#### REMEDIATION COMMENT

Addressed in: [🔗 e8d80d7](#)

**SOLVED:** The `Moonwell` team solved the issue in the specified commit id by re-pricing the redeem payout on the simulated post crystallise book whenever the fee crystallise defers.

---

## Strategy becomes permanently un-leverageable once tokenId reaches 0 in Executed state, forcing full settlement + redeployment

• Low · 2.5 A0:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

### DESCRIPTION

`LeveragedAerodromeCLStrategy.deployIdle()` is the only entrypoint outside the one-shot `execute()` that can move idle USDC into the levered concentrated-liquidity position. It delegates to `LeveragedAeroManager.deployIdleImpl()`, which sizes the borrowed legs and hands them to `_wrapAddRestake()`:

SOL 1297-1299

```
1297 | uint256 tokenId_ = $.tokenId;           // <-- reads the stored NFT id
1298 | address gauge_ = $.gauge;
1299 | ICLGauge(gauge_).withdraw(tokenId_);    // <-- reverts if tokenId_ == 0
```

`_wrapAddRestake()` operates unconditionally on the stored `$.tokenId`, and both `ICLGauge.withdraw()` and the subsequent `INonfungiblePositionManager.increaseLiquidity()` revert when that id is zero. However, no code path mints a fresh NFT while `_state == Executed`: `executeImpl()` is reachable only through `execute()`, which is gated on `State.Pending` and is one-shot; `rerangeImpl()` mints but early-returns on `if ($.tokenId == 0) return;` before reaching the mint; `compound()` early-returns; and `adjustLeverage()` -up routes back into `_wrapAddRestake()`. Once `$.tokenId` is zeroed while the strategy remains `Executed`, the levered position cannot be re-established without a full lifecycle reset.

This is not a direct-attack finding, there is no privilege escalation and no theft, but the bricked state is reachable through routine flow with no attacker-controlled input, driven by

`LeveragedAeroManager.redeemUnwindImpl()`, whose full-redemption branch clears position state while leaving `_state == Executed`:

SOL 316-321

```
316 | if (shares == supply) {
317 |     ...
318 |     $.tokenId = 0;
319 |     $.postTickLower = 0;
320 |
321 | }
```

```
    $.postTickUpper = 0;  
}
```

The comparison uses post-crystallise `supply`, so the branch fires whenever `feeRecipient` holds no residual balance at fulfil time, which occurs under zero-fee configuration, after `feeRecipient` exits its own accumulated shares, under a deferred crystallisation caused by `setOpenDeposits(false)`, or idempotently on an already-flat book.

#### RECOMMENDATION

Give `deployIdleImpl()`, and the leverage-up branch of `adjustLeverage()`, a path to mint a fresh NFT when `$.tokenId == 0`, so the flat-book `Executed` state is recoverable in place.

#### REMEDIATION COMMENT

**RISK ACCEPTED:** The `Moonwell` team made a business decision to risk accept this finding and not alter the contracts, stating that:

The premise does not hold: `redeploy(minLiquidity)` is a proposer entrypoint that requires exactly `Executed + tokenId == 0` and re-runs the genesis mint over the idle balance — precisely the path the remediation asks for, documented for operators and covered by the venue-migration suite. A flat `Executed` book is a supported, NAV-priceable state that `flatten()` produces deliberately.

# Asymmetric virtual-offset between deposit and redeem deviates from ERC-4626 canonical accounting

● Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

## DESCRIPTION

`LeveragedAerodromeCLStrategy.deposit()` prices shares using an OpenZeppelin-style virtual offset:

SOL 963

```
963 | shares = Math.mulDiv(assets, supply + SHARES_VIRTUAL_OFFSET, navNet + 1); // K = 1e6
```

However, the paired `redeem()` prices assets with the un-offset formula:

SOL 1194

```
1194 | assetsOut = Math.mulDiv(shares, navNet, supply); // rounds down, LP-favourable
```

The constant is documented as an ERC-4626 virtual share offset matching the vault's `_decimalsOffset()` (USDC 6dp → `1e6`), so the stated intent is alignment with OZ ERC-4626, whose canonical accounting applies the offset symmetrically in both directions, `_convertToShares = mulDiv(a, S + 10decOff, N + 1)` and `_convertToAssets = mulDiv(s, N + 1, S + 10decOff)`. The strategy applies it on deposit only. `previewRedeem()` mirrors the on-chain un-offset formula, so strategy-level quotes remain internally consistent; the divergence surfaces against the vault's own ERC-4626 surface.

## RECOMMENDATION

Restore symmetry with OZ ERC-4626 by applying the virtual offset on the redemption side as done on deposit side.

## REMEDIATION COMMENT

**RISK ACCEPTED:** The `Moonwell` team made a business decision to risk accept this finding and not alter the contracts, stating that:

The vault is a plain ERC-20 share ledger, not an ERC-4626 vault — there is no canonical accounting to deviate from (a stale comment referencing a non-existent `_decimalsOffset()` invited the finding and has been corrected). The offset guards issuance only, and the inflation attack is closed structurally (owner seed mint precedes any public deposit; round-to-zero deposits revert). The round-trip bias is bounded in closed form at  $(G - 1)$  micro-USDC at any deposit size; a symmetric offset would ratchet the performance HWM on rounding crumbs, desync the fast and async exits, and strand NAV behind the last redeemer.

---

# Full-redeem branch passes an unbounded amountInMax to \_redeemCoverShortfall, exposing the last redeemer to MEV extraction

• Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

## DESCRIPTION

The full-redeem branch of `LeveragedAeroManager.redeemUnwindImpl()` covers per-leg impermanent-loss shortfalls by calling `_redeemCoverShortfall()` with `type(uint256).max` as the caller-supplied ceiling:

SOL 321-326

```
321 | if (shares == supply) {
322 |     // Phase 1 (oracle-free): cover IL shortfall from idle USDC via exact-output swap.
323 |     if (cbShort > 0) _redeemCoverShortfall($.cbBTC, $.mCbBTC, cbShort,
    |     type(uint256).max);
324 |     if (wethShort > 0) _redeemCoverShortfall($.weth, $.mWeth, wethShort,
    |     type(uint256).max);
325 |     ...
326 | }
```

Inside `_redeemCoverShortfall()` the only surviving bound is `maxIn = min(usdcBal, amountInMax)`, which with an unbounded ceiling resolves to the strategy's entire idle USDC balance. There is no per-swap oracle bound, and the redeem path deliberately does not calm-gate.

However, the permissionless deleverage path routes the same primitive through `_rebalanceCover()` and does apply an oracle-and-slippage ceiling, computing `buyMax = _tokenToUsdc(shortAmt, dec, pDeficit, pUsdc) × (10000 + maxSlippageBps) / 10000` and passing that instead. Same helper, same configuration, applied on one path and omitted on the other. The residual leg sweeps at compound the exposure, calling `_sweepLegToUsdc()` on both legs with `minOut = 0`, so every intra-redeem DEX interaction on this branch is either uncapped on the buy or unfloored on the sell.

The branch is reachable through `redeem()` when `shares == totalSupply`, the last remaining holder self-exiting, and through `fulfillRedeem()` or `emergencyRedeem()` on a request whose shares equal total supply at fulfil time. The attacker is any MEV searcher watching the mempool; backend fulfilments broadcast at a known-timed moment, making the target transaction easy to bundle around.

#### RECOMMENDATION

Wrap the oracle read in a `try/catch`: on success, set `amountInMax = min(type(uint256).max, _tokenToUsdc(shortAmt, ...) * (10000 + maxSlippageBps) / 10000)`; on failure (stale/unavailable feed), fall back to `type(uint256).max` so the last-holder full-exit is never blocked by an oracle outage, closing the MEV window whenever a live feed is available while preserving the documented oracle-free fail-safe.

#### REMEDIATION COMMENT

**RISK ACCEPTED:** The `Moonwell` team made a business decision to risk accept this finding and not alter the contracts, stating that:

The branch is deliberately balance-bounded per swap and floor-bounded in aggregate: it is the body of the oracle-free exit, and a per-swap Chainlink ceiling would make the trustless deadman fail in the outage it exists to survive (the suggested try/catch fallback is not constructible around the in-library read). Reachability is also narrower than reported — the fast redeem() never runs this branch; only the proposer's fulfillRedeem and the time-gated emergencyRedeem do, both behind the aggregate payout floor that finding 14's fresh-floor fix strengthens.

---

# DegenerateRange revert in deployIdleImpl cascades into compound, bricking harvest and interest hedging during volatility

● Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

## DESCRIPTION

`LeveragedAeroManager.deployIdleImpl()` sizes its supply, borrow, and LP-add against the stored position range rather than a freshly centred one:

SQL 417-418

```
417 (uint256 cbBTCamt, uint256 wethAmt) = _supplyAndBorrow(amount, $.posTickLower,  
418 $.posTickUpper);  
    //                                     ↑ STORED range, not  
    centred
```

In asset-mode, where `$.cbBTC == $.usdc`, so the leg-B slot is itself the unit of account, as in the intended WETH/USDC configuration, that sizing flows through

`LeveragedAeroValuation.assetModeSplit()` into `_rangeRatio()`, which reads the pool's live `sqrtP` and requires both leg amounts to be non-zero:

SQL 767-768

```
767 (needA, needU) = legAIsToken0 ? (amt0, amt1) : (amt1, amt0);  
768 if (needA == 0 || needU == 0) revert DegenerateRange();
```

Once the pool's tick leaves `[posTickLower, posTickUpper]`, `getAmountsForLiquidity()` returns one side as zero and `DegenerateRange` fires.

However, `compoundImpl()` tail-calls `deployIdleImpl()` and inherits the revert, and the three preceding steps of compound all mutate state: `getReward()` claims AERO from the gauge, `swapAeroToUsdc()` performs an external DEX swap, and `_hedgeInterestDrift()` repays borrowed WETH on Moonwell. Because step 6 reverts, all of them are unwound atomically and none persist. Unlike `deployIdle()`, which a proposer can defer without harm, compound is time-sensitive: unswapped AERO earns nothing further, and unhedged interest widens the delta-neutral leg's exposure every block. The hedge basis is itself a documented invariant, the header comment notes that without

that step every harvest leaves the book a little more short with nothing ever removing it, and this cascade violates it during precisely the worst moments.

#### RECOMMENDATION

Decouple compound's yield-and-hedge work from the redeploy step's ability to succeed: yield realisation and interest hedging should persist whenever pool state permits, and only the redeploy should defer.

#### REMEDATION COMMENT

**RISK ACCEPTED:** The **Moonwell** team made a business decision to risk accept this finding and not alter the contracts, stating that:

The atomicity is the codebase's resumable-op posture, and the deferral costs nothing measurable: an out-of-range position accrues no new gauge emissions (the unclaimed tranche is static, not decaying), and the interest-hedge measure is cumulative — the next successful harvest resumes exactly where the reverted one stopped. DegenerateRange is one of several fail-closed conditions on that path (a best-effort catch would silently skip the calm gate, the liquidity floor, and the health assert), and the recovery routes — rerange onto spot, or flatten + redeploy, which itself realises the tranche and zeroes the drift — are now pinned by a regression test.

---

HAL-021

SOLVED

## AERO harvest slippage floor in compoundImpl ignores the USDC/USD peg feed

● Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

### DESCRIPTION

`LeveragedAeroManager.compoundImpl()` computes the minimum acceptable USDC output for the AERO harvest swap directly from the AERO/USD feed:

SOL 453

```
453 | uint256 floor = Math.mulDiv(aeroBal, _readUsd8($.aeroUsdFeed), 1e20) * (10000 -  
    | maxSlippageBps) / 10000;
```

The decimal derivation is `aeroBal(18dp) × aeroUsdPrice(8dp) / 1e20`, which yields a value denominated in **USD at 6 decimals**, dollars, not USDC face. The result is then passed as the floor against `swapAeroToUsdc()`, whose return value is a balance delta measured in USDC face. The expression therefore treats one USDC as one dollar unconditionally.

However, the rest of the codebase does not make that assumption. Both `nav()` and `_hedgeLeg()` perform the full USD-to-USDC-face conversion through `pUsdc`, using `_usdcValue()`. Only the compound floor omits it. While USDC holds its peg the two are indistinguishable, so the defect is invisible in normal operation and surfaces exactly when the peg breaks.

### RECOMMENDATION

Convert the USD-denominated value into USDC face before applying the slippage haircut, so the floor is expressed in the same units as the balance delta it is compared against.

### REMEDIATION COMMENT

Addressed in: [🔗 2e4de48](#)

**SOLVED:** The [Moonwell](#) team solved the issue in the specified commit id by following the mentioned recommendation.

---

HAL-022

SOLVED

## settleImpl sells both LP legs but never liquidates the claimed reward token, stranding it outside every holder's redemption basis

● Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:L

### DESCRIPTION

`LeveragedAeroManager.settleImpl()` liquidates the position at settlement by making exactly two `_swapTokenToUsdc()` calls in step 5, covering `$.weth` and `$.cbBTC`. There is no `swapAeroToUsdc()` call anywhere in `settleImpl()`, that helper is invoked only from `compoundImpl()`, and `_settle()` finishes with `_pushAllToVault($.usdc)`, which moves USDC alone.

However, the settlement itself claims AERO: the `gauge.withdraw()` performed during the unwind pulls the accrued reward tranche into the strategy. That balance, together with everything accrued since the last successful `compound()`, is therefore stranded at the moment the strategy transitions to `Settled`. Nothing sells it and nothing pushes it to the vault.

The codebase treats this as recoverable rather than distributable. `rescueToVault()` guards on `(token == aero && _state != State.Settled)`, deliberately permitting an AERO sweep once settled, and its NatSpec states that post-settlement the balance is a stray and sweeping it to the vault, where the owner's non-asset `rescueERC20()` applies, is the only recovery. The value therefore exits through an owner-controlled rescue path rather than through the redemption basis.

### RECOMMENDATION

Sell the reward token during settlement rather than leaving it for rescue.

### REMEDIATION COMMENT

Addressed in: [🔗 eb129f4](#)

**SOLVED:** The [Moonwell](#) team solved the issue in the specified commit id by following the mentioned recommendation.

---

# emergencyRedeem reads Chainlink through \_settleShortfall, bricking the deadman in the exact outage it exists to survive

• Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

## DESCRIPTION

`LeveragedAerodromeCLStrategy.emergencyRedeem()` is documented as the trustless deadman backstop, on the stated reasoning that a single gate covers the whole deadman matrix: fulfilment is oracle-free, so an oracle outage with a live backend resolves through normal `fulfillRedeem()`, and the only genuinely stuck case, oracle down **and** backend dead, self-resolves here. The path is built to run with `nav()` under `try/catch` and to use only oracle-free primitives inside the unwind: pool `sqrtp`, borrow balances, and collected token amounts.

However, that contract is violated on the full-redeem branch when idle USDC is zero and impermanent loss has created a leg shortfall. Tracing `emergencyRedeem()` → `_proportionalRedeem()` → `redeemUnwindImpl()`, every step is oracle-free until the branch:

SOL 316-324

```

316 | if (shares == supply) {
317 |     // Phase 1 (oracle-free): cover IL shortfall from idle USDC via exact-output swap.
318 |     if (cbShort > 0) _redeemCoverShortfall($.cbBTC, $.mCbBTC, cbShort,
type(uint256).max);
319 |     if (wethShort > 0) _redeemCoverShortfall($.weth, $.mWeth, wethShort,
type(uint256).max);
320 |     // Phase 2 (self-fund fallback): if residual debt remains after Phase 1 (idle == 0
case),
321 |     _settleShortfall();
322 |     _redeemCollateral(shares, supply);
323 |     ...
324 | }
```

`_redeemCoverShortfall()` short-circuits cleanly on `if (usdcBal == 0 || amountOut == 0) return;` no revert, but no debt repaid either, so the residual stays on Moonwell. Control then reaches `_settleShortfall()` (:1132), whose only early exit is the no-debt-at-all gate `if (cbDebtRem == 0 && wethDebtRem == 0) return;`. Past that gate it calls `_readAllPrices()`, which reads the cbBTC, WETH, and USDC feeds unconditionally. Any one of them being stale bubbles `ChainlinkReader.StaleOracle` up the entire call stack.

The revert lands after the holder has already invoked the deadman and before any collateral can be redeemed. The request remains open with `r.settled == false`, and there is no on-chain route to withdraw until the feed recovers: `redeem()` reads `nav()` and is oracle-dependent, `fulfillRedeem()` is proposer-gated with the backend dead by assumption, and `emergencyRedeem()` is the path this finding shows fails. `cancelRedeem()` returns the shares to the holder's wallet, but shares are not principal, they remain unredeemable through every available path.

#### RECOMMENDATION

Correct the documentation so it stops asserting a guarantee the code does not provide. The NatSpec at `emergencyRedeem()` states that the only stuck case, oracle down and backend dead, self-resolves there, and the branch comment describes Phase 2 as a self-funding fallback without noting that it is oracle-dependent. Both should state plainly that the path is oracle-free only while Phase 1 fully covers the shortfall, and that a full redeem with zero idle USDC and a non-zero leg shortfall requires live Chainlink feeds. Holders and integrators can then reason about the actual failure surface rather than a promised one.

#### REMEDIATION COMMENT

Addressed in: [🔗 e52190d](#)

**SOLVED:** The [Moonwell](#) team solved the issue in the specified commit id by following the mentioned recommendation.

---

HAL-024

SOLVED

## Donating a single share to the vault permanently disables the asset rescue safety valve

● Low · 2.5

A0:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

### DESCRIPTION

`LeveragedAeroVault` is a plain OpenZeppelin ERC20 with no `_update()` or `_beforeTokenTransfer()` override blocking self-transfers, so its own address is a valid transfer target. Any holder can call `transfer(address(vault), 1)` and park 1 wei share on the vault itself.

However, no path exists to reduce those shares. `redeemSettled()` burns only from `msg.sender` and never touches the vault's own balance, and `rescueERC20()` explicitly rejects `token == address(this)`, so vault-held shares are unreachable through the rescue path. Meanwhile the asset rescue gate requires `totalSupply() == 0`:

SOL 311

```
311 | require(token != asset || totalSupply() == 0, "LAV: asset reserved for redemptions");
```

A single donated share therefore pins `totalSupply()` above zero permanently and disables the owner's asset rescue for the life of the contract.

### RECOMMENDATION

Relax the rescue gate to disregard vault-owned shares, requiring `totalSupply() == balanceOf(address(this))` rather than `totalSupply() == 0`. The intended semantics are preserved exactly, while any external holder retains shares, the pot stays reserved, and the gate becomes satisfiable once every legitimate holder has redeemed regardless of what has been donated.

### REMEDIATION COMMENT

Addressed in: [🔗 b9cad39](#)

**SOLVED:** The `Moonwell` team solved the issue in the specified commit id by following the mentioned recommendation.

## Single shared maxDelay staleness threshold used across price feeds of differing volatility

• Informational · 1.7 A0:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

### DESCRIPTION

`LeveragedAerodromeCLStrategy` stores a single `Layout.maxDelay` value that is used as the Chainlink staleness threshold for every price feed the protocol reads, `cbBTCFeed`, `wethFeed`, `usdcFeed`, and `aeroUsdFeed` (all defined in `InitParams` / `Layout`). Every read funnels through `ChainlinkReader.readUsd`:

SOL 38-39

```
38 | uint256 age = block.timestamp > updatedAt ? block.timestamp - updatedAt : 0;  
39 | if (age > maxDelay) revert StaleOracle();
```

via call sites in `LeveragedAeroValuation.sol` and `LeveragedAeroManager.sol`. `maxDelay` is set exactly once, in the initializer, bounded only by `if (p.maxDelay == 0 || p.maxDelay > 7 days) revert OracleParamOutOfRange();`; there is no setter and no per-feed override; changing it requires a full contract upgrade. On Base, the underlying Chainlink feeds have materially different heartbeat/deviation configurations: cbBTC/USD and WETH/USD update roughly hourly with a ~0.5% deviation trigger, while USDC/USD updates roughly every 24 hours with a ~0.1% deviation trigger, and AERO/USD is thinner and less frequently updated still. Because all four feeds are validated against the identical `maxDelay`, the value can never be correctly tuned for all of them at once.

### RECOMMENDATION

Replace the single `Layout.maxDelay` with a per-feed mapping (e.g. `mapping(address feed => uint256 maxDelay)`), set at initialization from per-feed values in `InitParams` and validated with the existing `(0, 7 days]` bound per entry, so `ChainlinkReader.readUsd` is called with the threshold appropriate to each feed's actual heartbeat/deviation configuration (tight for cbBTC/WETH, looser for USDC/AERO).

### REMEDIATION COMMENT

**ACKNOWLEDGED:** The `Moonwell` team made a business decision to acknowledge this finding and not alter the contracts, stating that:

maxDelay is a dead-feed detector, not a price-freshness bound — while a feed is alive, its deviation trigger governs drift, so a single threshold sized off the slowest feed admits no materially wrong

price; it only lengthens detection of a stopped feed. A per-feed mapping would require a coordinated edit across three parity-checked Layout copies, three new members in the owner-committed migration preimage, and a new valuation-config field, against a library near the EIP-170 cap. Instead the deployed value is being retuned down toward the slowest heartbeat and per-feed updatedAt liveness is monitored off-chain. Two report premises were also inaccurate: the leg/reward feeds are runtime-replaceable via the owner-staged migration, and there is no upgrade path at all (the threshold is fixed per clone).

HAL-026

SOLVED

## Proposer role in BaseStrategy is set permanently at initialize() with no rotation mechanism

• Informational · 1.7 A0:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

### DESCRIPTION

`BaseStrategy::initialize()` sets `_proposer = proposer_;` exactly once, and no function anywhere in `BaseStrategy.sol` or `LeveragedAerodromeCLStrategy.sol`, no `setProposer`, `updateProposer`, `rotateProposer`, or similar ever changes it afterward. This is explicit by design: `BaseStrategy.sol` documents that clones have "no migration path," and `LeveragedAeroVault.sol:44` similarly documents the strategy binding as "Set once, never rotated." The `proposer` role gates every operationally critical function on the strategy: `deployIdle`, `compound`, `rerange`, `adjustLeverage`, `fulfillRedeem`, and `updateParams`. Because the strategy is a non-upgradeable ERC-1167 clone (`BaseStrategy.sol`, "Designed for Clones," "Clones are not upgradeable", no `upgradeTo` / `upgradeToAndCall` exists on this contract family), there is no code path to replace a compromised or lost proposer key on a live clone

### RECOMMENDATION

Add an `onlyOwner` or `onlyVault`-gated `setProposer(address newProposer_)` function to `BaseStrategy` so a compromised or lost proposer key can be rotated without unwinding the entire position via `settleStrategy()`.

### REMEDIATION COMMENT

Addressed in: [🔗 dd68c1e](#)

**SOLVED:** The `Moonwell` team solved the issue in the specified commit id by following the mentioned recommendation.



## Discarded Comptroller error code on the before `getAccountLiquidity()` snapshot in `deleverageImpl()`

• Informational · 1.7 A0:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

### DESCRIPTION

`LeveragedAeroManager::deleverageImpl()` takes its "before" shortfall snapshot by discarding the Comptroller's returned error code:

SOL

```
(,, uint256 shortfallBefore) = IComptroller($.comptroller).getAccountLiquidity(address(this));
```

If the Comptroller returns a non-zero error on this call, `shortfallBefore` is left as an unreliable, potentially-zero value with nothing in the function checking for that failure. This is inconsistent with the function's own second call to the same interface, taken after the deleverage action, which does capture and check the error code:

SOL

```
(uint256 err,, uint256 shortfallAfter) =  
IComptroller($.comptroller).getAccountLiquidity(address(this));
```

and gates the recovery check on it:

SOL

```
if (err != 0 || (shortfallAfter != 0 && shortfallAfter >= shortfallBefore)) {  
    revert UnhealthyPosition(healthAfter, minHealth);  
}
```

The same interface is called correctly elsewhere in the file, `_assertHealthy()` captures `err` and reverts with `UnhealthyPosition` if `err != 0`. So this file already has the correct pattern in two of its three `getAccountLiquidity()` call sites; only the "before" read in `deleverageImpl()` omits it. In practice, the blast radius is narrow: the `shortfallAfter >= shortfallBefore` comparison is only

reached when `shortfallAfter != 0`, and a separate strict-improvement gate (`if (healthAfter <= healthBefore) revert UnhealthyPosition(...)`) also constrains the outcome.

#### RECOMMENDATION

Capture and check the error code on the first call the same way the second call already does.

#### REMEDATION COMMENT

**ACKNOWLEDGED:** The `Moonwell` team made a business decision to acknowledge this finding and not alter the contracts, stating that:

Provably fail-safe: Compound-lineage `getAccountLiquidity` returns `(err, 0, 0)` on every error branch, so a failed before-read leaves `shortfallBefore == 0` and makes the recovery gate strictly tighter; and the error condition cannot clear mid-transaction (nothing exits a market), so the checked after-read reverts regardless. Capturing the code would spend scarce bytecode to guard an unreachable branch of the permissionless safety valve. The asymmetry is now documented in place, and the previously-untested shortfall arm of the gate is pinned by two new tests.

---

## settleImpl() unconditionally requires all three Chainlink feeds to be live, blocking settlement during an oracle outage even when no swap is needed

• Informational · 1.7 A0:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

### DESCRIPTION

`LeveragedAeroManager::settleImpl()` reads all three price feeds unconditionally, before any check of whether there is actually anything to sweep:

#### SOL

```
// 5. Sweep residual WETH + cbBTC → USDC (Chainlink-bounded min-out)
{
    (uint256 pBTC, uint256 pETH, uint256 pUsdc) = _readAllPrices();
```

`_readAllPrices()` calls `_readUsd8()` for `cbBTCFeed`, `wethFeed`, and `usdcFeed` in sequence, each going through `LeveragedAeroValuation.readUsd8()` into `ChainlinkReader.readUsd()`, which reverts on `SequencerDown()`, `GracePeriodNotOver()`, or `StaleOracle()`. The only balance short-circuit in this flow is `if (bal <= keep) return;` inside `_sweepLegToUsdc()`, but that check runs *after* the price read has already executed, so even a strategy holding zero WETH/cbBTC (nothing to swap at all) still reverts if any one of the three feeds is stale or the sequencer is down.

`maxDelay` is configurable up to 7 days, so an outage can hold this window open for a long time by design. Since `BaseStrategy::settle()` writes `_state = State.Settled` and then calls into `settleImpl()` in the same call frame, a revert there reverts the whole transaction and leaves state unchanged; correspondingly `LeveragedAeroVault::settleStrategy()` only sets `settled = true` after `IStrategy(strategy_).settle()` returns successfully. `redeemSettled()` gates on that exact flag (`require(settled, "LAV: not settled");`). No fallback exists: there is no force-settle path, no admin price override, and `_updateParams` is a no-op for the feed addresses/`maxDelay`, a dead feed cannot be swapped out at runtime, only via a full contract upgrade of the (non-upgradeable, per earlier findings) clone family.

### RECOMMENDATION

Skip the price read entirely when there is nothing to sweep: check

```
IERC20($.weth).balanceOf(address(this)) == 0 &&
IERC20($.cbBTC).balanceOf(address(this)) == 0 before calling _readAllPrices().
```

#### REMEDIATION COMMENT

**ACKNOWLEDGED:** The **Moonwell** team made a business decision to acknowledge this finding and not alter the contracts, stating that:

The guard would fire in exactly one state — an already-flat book — because reaching the sweep with both legs empty also requires zero residual debt, and any real unwind of a levered book leaves one or the other. In that state nothing is impaired: flat-book nav() is oracle-free, the redeem valves are oracle-free, migration reads no oracle, and the owner's settle is retryable. The report's suggested patch is also inert in the shipping asset-mode configuration, where the leg-B slot is USDC and the tested balance is the idle USDC the collateral redeem just produced — the correct form, should this ever be revisited, is documented at the call site.

---

HAL-029

SOLVED

# Missing bps cap in performanceFeeShares underflows and silently defers all fee crystallisation, enabling an owner-triggered fee-shift window

• Informational · 0.5 A0:S/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

## DESCRIPTION

`LeveragedAeroFees.performanceFeeShares()` computes the performance-fee slice as `mulDiv(totalGainUsdc - protocolUsdc, performanceFeeBps, 10_000)`, where `protocolUsdc = mulDiv(totalGainUsdc, protocolFeeBps, 10_000)`. The only bound applied to `protocolUsdc` is a ceiling against `navPre`:

SQL 160-163

```
160 protocolUsdc = Math.mulDiv(totalGainUsdc, protocolFeeBps, 10_000);
161 if (protocolUsdc > navPre) protocolUsdc = navPre; // ← navPre cap only; NOT
    totalGainUsdc
162 ...
163 uint256 feeValueUsdc = Math.mulDiv(totalGainUsdc - protocolUsdc, performanceFeeBps,
    10_000);
```

However, if `protocolFeeBps > 10_000` then `protocolUsdc > totalGainUsdc` in the common case of a small gain relative to NAV, the `navPre` ceiling never fires because gain is far below NAV, and the subsequent subtraction panics with `0x11` on arithmetic underflow.

## RECOMMENDATION

Clamp the input inside `performanceFeeShares()` before any arithmetic runs, so that a `protocolFeeBps` above `10_000` is treated as 100% of gain rather than propagating into a subtraction that can underflow.

## REMEDIATION COMMENT

Addressed in: [f6849fc](#)

**SOLVED:** The [Moonwell](#) team solved the issue in the specified commit id by following the mentioned recommendation.

## Disclaimer

*Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.*